

Exp. No. 16 Write a C Program to Generate the Three address code representation for the given input statement.

```
#include<stdio.h>

#include<conio.h>

#include<stdlib.h>

#include<string.h>

struct three

{

char data[10],temp[7];

}s[30];

int main()

{

char d1[7],d2[7]="t";

int i=0,j=1,len=0;

FILE *f1,*f2;

//clrscr();

f1=fopen("sum.txt","r");

f2=fopen("out.txt","w");

while(fscanf(f1,"%s",s[len].data)!=EOF)

len++;

itoa(j,d1,7);

strcat(d2,d1);

strcpy(s[j].temp,d2);

strcpy(d1,"");

strcpy(d2,"t");

if(!strcmp(s[3].data,"+"))

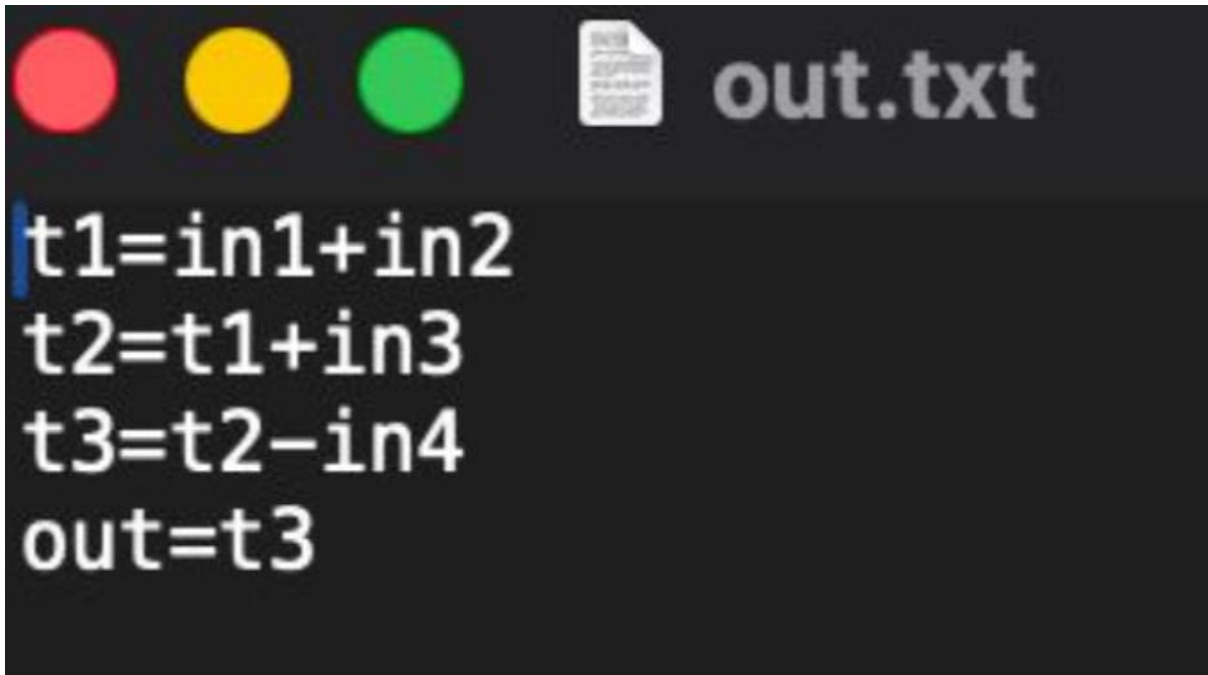
{
```

```

fprintf(f2,"%s=%s+%s",s[j].temp,s[i+2].data,s[i+4].data);
j++;
}
else if(!strcmp(s[3].data,"-"))
{
fprintf(f2,"%s=%s-%s",s[j].temp,s[i+2].data,s[i+4].data);
j++;
}
for(i=4;i<len-2;i+=2)
{
itoa(j,d1,7);
strcat(d2,d1);
strcpy(s[j].temp,d2);
if(!strcmp(s[i+1].data,"+"))
fprintf(f2,"\n%s=%s+%s",s[j].temp,s[j-1].temp,s[i+2].data);
else if(!strcmp(s[i+1].data,"-"))
fprintf(f2,"\n%s=%s-%s",s[j].temp,s[j-1].temp,s[i+2].data);
strcpy(d1,"");
strcpy(d2,"t");
j++;
}
fprintf(f2,"\n%s=%s",s[0].data,s[j-1].temp);
fclose(f1);
fclose(f2);
getch();
}

```

OUTPUT:



```
t1=in1+in2
t2=t1+in3
t3=t2-in4
out=t3
```

Exp. No. 17 Write a C program for implementing a Lexical Analyzer to Scan and Count the number of characters, words, and lines in a file.

```
#include <stdio.h>

int main()
{
    char str[100]; //input string with size 100
    int words=0, newline=0, characters=0; // counter variables
    scanf("%[^~]", &str); //scanf formatting
    for(int i=0; str[i]!='\0'; i++)
    {
        if(str[i] == ' ')
        {
            words++;
        }
        else if(str[i] == '\n')
        {

```

```
newline++;  
words++;//since with every next line new words start. corner case 1  
}  
else if(str[i] != ' ' && str[i] != '\n'){  
characters++;  
}  
}  
if(characters > 0)//Corner case 2,3.  
{  
words++;  
newline++;  
}  
printf("Total number of words : %d\n",words);  
printf("Total number of lines : %d\n",newline);  
printf("Total number of characters : %d\n",characters);  
return 0;  
}
```

OUTPUT:

```
C:\Users\Administrator\Documents\exp 17.exe
void main()
{
int a;
int b;
a = b + c;
c = d * e;

}~
Total number of words : 19
Total number of lines : 8
Total number of characters : 34

-----
Process exited after 121.5 seconds with return value 0
Press any key to continue . . .
```

Exp. No. 18 Write a C program to implement the back end of the compiler.

```
#include<stdio.h>

#include<conio.h>

#include<string.h>

int main()

{

int n,i,j;

char a[50][50];

printf("enter the no: intermediate code:");

scanf("%d",&n);

for(i=0;i<n;i++)

{

printf("enter the 3 address code:%d:",i+1);

for(j=0;j<6;j++)

{

scanf("%c",&a[i][j]);
```

```
}  
  
}  
  
printf("the generated code is:");  
for(i=0;i<n;i++)  
{  
    printf("\n mov %c,R%d",a[i][3],i);  
    if(a[i][4]=='-')  
    {  
        printf("\n sub %c,R%d",a[i][5],i);  
    }  
    if(a[i][4]=='+')  
    {  
        printf("\n add %c,R%d",a[i][5],i);  
    }  
    if(a[i][4]=='*')  
    {  
        printf("\n mul %c,R%d",a[i][5],i);  
    }  
    if(a[i][4]=='/')  
    {  
        printf("\n div %c,R%d",a[i][5],i);  
    }  
    printf("\n mov R%d,%c",i,a[i][1]);  
    printf("\n");  
}  
return 0;  
}
```

OUTPUT:

```
C:\Users\Administrator\Documents\exp 18.exe
enter the no: intermediate code:2
enter the 3 address code:1:a=b+c
enter the 3 address code:2:d=n*d
the generated code is:
mov b,R0
add c,R0
mov R0,a

mov n,R1
mul d,R1
mov R1,d

-----
Process exited after 51.99 seconds with return value 0
Press any key to continue . . .
```

Exp. No. 19 Write a C program to compute LEADING() – operator precedence parser for the given grammar $E \rightarrow E + T \mid T T \rightarrow T * F \mid F F \rightarrow (E) \mid$

```
#include<conio.h>
```

```
#include<stdio.h>
```

```
char arr[18][3]={{{'E', '+', 'F'},{'E', '*', 'F'},{'E', '(', 'F'},{'E', ')', 'F'},{'E', 'i', 'F'},{'E', '$', 'F'},
{'F', '+', 'F'},{'F', '*', 'F'},{'F', '(', 'F'},{'F', ')', 'F'},{'F', 'i', 'F'},{'F', '$', 'F'},{'T', '+', 'F'},
{'T', '*', 'F'},{'T', '(', 'F'},{'T', ')', 'F'},{'T', 'i', 'F'},{'T', '$', 'F'}};
```

```
char prod[] = "EETTFF";
```

```
char res[6][3]={{{'E', '+', 'T'},{'T', '\0'},{'T', '*', 'F'},{'F', '\0'},{'(', 'E', ')'},{'i', '\0'}};
```

```
char stack [5][2];
```

```
int top = -1;
```

```
void install(char pro, char re) {
```

```
    int i;
```

```
    for (i = 0; i < 18; ++i) {
```

```
        if (arr[i][0] == pro && arr[i][1] == re) {
```

```
            arr[i][2] = 'T';
```

```

break;

}

}

++top;

stack[top][0] = pro;

stack[top][1] = re;

}

int main() {

int i = 0, j;

char pro, re, pri = ' ';

for (i = 0; i < 6; ++i) {

for (j = 0; j < 3 && res[i][j] != '\0'; ++j) {

if (res[i][j] == '+' || res[i][j] == '*' || res[i][j] == '(' || res[i][j] == ')' || res[i][j] ==

'i' || res[i][j] == '$') {

install(prod[i], res[i][j]);

break;

}

}

}

while (top >= 0) {

pro = stack[top][0];

re = stack[top][1];

--top;

for (i = 0; i < 6; ++i) {

if (res[i][0] == pro && res[i][0] != prod[i]) {

install(prod[i], re);

}

}

}

```



```
}  
  
for (i = 0; i < 18; ++i) {  
    printf("\n\t");  
    for (j = 0; j < 3; ++j)  
        printf("%c\t", arr[i][j]);  
    }  
    getch();  
    printf("\n\n");  
    for (i = 0; i < 18; ++i) {  
        if (pri != arr[i][0]) {  
            pri = arr[i][0];  
            printf("\n\t%c -> ", pri);  
        }  
        if (arr[i][2] == 'T')  
            printf("%c ", arr[i][1]);  
        }  
        getch();  
    }
```

OUTPUT:

```

C:\Users\Administrator\Documents\exp 19.exe

E      +      T
E      *      T
E      (      T
E      )      F
E      i      T
E      $      F
F      +      F
F      *      F
F      (      T
F      )      F
F      i      T
F      $      F
T      +      F
T      *      T
T      (      T
T      )      F
T      i      T
T      $      F

E -> + * ( i
F -> ( i
T -> * ( i
-----
Process exited after 55.15 seconds with return value 0
Press any key to continue . . .

```

Exp. No. 20 Write a C program to compute TRAILING() – operator precedence parser for the given grammar $E \rightarrow E + T \mid T T \rightarrow T * F \mid F F \rightarrow (E) \mid id$

```

#include<conio.h>

#include<stdio.h>

char arr[18][3]={{'E', '+', 'F'}, {'E', '*', 'F'}, {'E', '(', 'F'}, {'E', ')', 'F'}, {'E', 'i', 'F'},
{'E', '$', 'F'}, {'F', '+', 'F'}, {'F', '*', 'F'}, {'F', '(', 'F'}, {'F', ')', 'F'}, {'F', 'i', 'F'},
{'F', '$', 'F'}, {'T', '+', 'F'}, {'T', '*', 'F'}, {'T', '(', 'F'}, {'T', ')', 'F'}, {'T', 'i', 'F'},
{'T', '$', 'F'},
};

char prod[6] = "EETFFF";

char res[6][3]={ {'E', '+', 'T'}, {'T', '\0', '\0'}, {'T', '*', 'F'}, {'F', '\0', '\0'}, {'(', 'E',

```

```

    '}', {'i', '\0', '\0'},});

char stack[5][2];

int top = -1;

void install(char pro, char re) {

    int i;

    for (i = 0; i < 18; ++i) {

        if (arr[i][0] == pro && arr[i][1] == re) {

            ++top;

            arr[i][2] = 'T';

            stack[top][0] = pro;

            stack[top][1] = re;

            break; // Added break to exit the loop when the match is found

        }

    }

}

int main() {

    int i = 0, j;

    char pro, re, pri = ' ';

    for (i = 0; i < 6; ++i) {

        for (j = 2; j >= 0; --j) {

            if (res[i][j] == '+' || res[i][j] == '*' || res[i][j] == '(' || res[i][j] == ')' || res[i][j] ==

            'i' || res[i][j] == '$') {

                install(prod[i], res[i][j]);

                break;

            } else if (res[i][j] == 'E' || res[i][j] == 'F' || res[i][j] == 'T') {

                if (res[i][j - 1] == '+' || res[i][j - 1] == '*' || res[i][j - 1] == '(' || res[i][j -

                1] == ')' || res[i][j - 1] == 'i' || res[i][j - 1] == '$') {

```

```

install(prod[i], res[i][j - 1]);

break;

}

}

}

}

while (top >= 0) {

pro = stack[top][0];

re = stack[top][1];

--top;

for (i = 0; i < 6; ++i) {

for (j = 2; j >= 0; --j) {

if (res[i][0] == pro && res[i][0] != prod[i]) {

install(prod[i], re);

break;

} else if (res[i][0] != '\0') break;

}

}

}

for (i = 0; i < 18; ++i) {

printf("\n\t");

for (j = 0; j < 3; ++j)

printf("%c\t", arr[i][j]);

}

printf("\n\n");

for (i = 0; i < 18; ++i) {

if (pri != arr[i][0]) {

pri = arr[i][0];

```

```

printf("\n\t%c -> ", pri);
}
if (arr[i][2] == 'T')
printf("%c ", arr[i][1]);
}

```

OUTPUT:

```

C:\Users\Administrator\Documents\exp 20.exe

E      +      T
E      *      T
E      (      F
E      )      T
E      i      F
E      $      F
F      +      F
F      *      F
F      (      F
F      )      T
F      i      F
F      $      F
T      +      F
T      *      T
T      (      F
T      )      T
T      i      F
T      $      F

E -> + * )
F -> )
T -> * )
-----
Process exited after 6.306 seconds with return value 0
Press any key to continue . . .

```